Luciteria Collector Cabinet — Phase 2 Design Document

Status: Design / Planning
Owner: Product + Engineering
Last updated: 2026-06-03
Scope: Four Phase 2 modules — (1) Enhanced Onboarding & Goal Setting, (2) Admin Dashboard Expansion, (3) Notification System, (4) Open Wishlist & Element Notes.

---

0. Context & Guiding Principles

Phase 1 delivered the "Collection-First" refactor: unified collection state tracking (`CollectionItem`), the interactive 118-element periodic table, the milestone engine (`Milestone`), gamified progress, a goal-oriented onboarding flow, contextual commerce, the dashboard, and Lucite Pro subscription management.

Phase 2 builds on this foundation to close four gaps:

1. **Onboarding goals are too completion-focused.** Real users have diverse motivations — inventory tracking, social sharing, acquisition, financial tracking, and casual discovery. The current Step 2 only offers "Complete Full Set / Complete a Format / Reach 50 / Reach 25 / Just Exploring."
2. **Admin has analytics but no operational intelligence.** There is no Shopify↔eBay SKU reconciliation, no demand/restock prioritization, and no UI to curate themed collection sets.
3. **Notifications are stubs.** `notifications.server.js` logs to console but there is no preference model, no trigger taxonomy, and no in-app delivery.
4. **Wishlist is flat and ownership has no provenance.** There are no context labels on wishlist intent and no private collector logs (source, price, condition) on owned items.

Design principles

- **Additive, non-breaking.** Phase 2 extends existing Prisma models and adds new ones. No destructive migrations. All new columns are nullable or have defaults.
- **Reuse existing models.** `CollectionGoalV2` , `CollectionItem.notes` , `WishlistItem.notifyOnRestock` , and `NotificationLog` already exist and are leveraged rather than duplicated.
- **High-signal over high-volume.** Notifications must respect a strict signal budget (see §3.5).
- **Prototype parity.** New features must work against the in-memory `mock-db.server.js` AND map cleanly onto the Prisma schema for production.

Cross-cutting open questions

- **OQ-0.1:** Do customer-facing features key off the `User` model (auth/onboarding) or the `Customer` model (collection records/subscription)? Phase 1 uses `User` for onboarding/collection and `Customer` for subscription/admin. Phase 2 designs below note which model each feature attaches to. A reconciliation of these two identities is a known tech-debt item (tracked separately).
- **OQ-0.2:** eBay data is currently only available via uploaded CSV exports (see `/home/ubuntu/Shared/Uploads/eBay-all-active-listings-report-*.csv`). Is a live eBay API in-

tegration in scope for Phase 2, or is CSV import the v1 source of truth? (Assumption below: CSV import v1, API stub for v2.)

---

1. Module 1 — Enhanced Onboarding & Goal Setting

1.1 Problem & objective

Current onboarding Step 2 (`app/routes/onboarding.collection-goal.jsx`) frames every user as a completionist. The 5 motivations users actually express are:

#	Motivation (user words)	Internal category	Primary surface affected
1	“Keep track of what is in my existing collection”	`INVENTORY`	Collection grid, periodic table
2	“Share my collection easily with friends”	`SOCIAL`	Public wishlist/collection share
3	“Add missing items to my collection”	`ACQUISITION`	Missing/Shop view, recommendations
4	“Keep track of what I have invested in my collection”	`FINANCIAL`	Element Notes (price), portfolio stats
5	“Just exploring”	`DISCOVERY`	Lightweight, low-friction dashboard

Objective: Capture one or more motivations during onboarding and use them to personalize the dashboard, default views, and notification defaults — WITHOUT removing the existing completion-style goals (those become an optional sub-step for users who pick `ACQUISITION` or `INVENTORY`).

1.2 User stories

- **US-1.1** As a new user, I can select one primary motivation and optionally additional motivations so the app reflects why I’m here.
- **US-1.2** As an inventory-focused user, I land on the collection grid (not the shop) after onboarding.
- **US-1.3** As an acquisition-focused user, I’m prompted to set a concrete completion goal (full set / format / count) and land on the “Missing” view.
- **US-1.4** As a financial-focused user, my Element Notes default to showing the price/value column and I see portfolio totals.
- **US-1.5** As a social user, a public share link is generated and surfaced immediately.
- **US-1.6** As an explorer, I get a minimal, no-pressure dashboard and can set a goal later.
- **US-1.7** As any user, I can change my motivations later from Preferences without re-running onboarding.

1.3 User flow modifications (Step 2)

Replace the single completion-goal screen with a two-substep Step 2:

```
Step 2a — "Why are you here, {firstName}?" (motivation picker)
  → Multi-select cards for the 5 motivations.
  → Exactly one must be marked PRIMARY (radio behavior on the "make primary" toggle);
    others are secondary (checkbox behavior).
  → Continue enabled once >=1 selected.

Step 2b — Conditional completion goal (only shown if PRIMARY ∈ {ACQUISITION, INVENT-
ORY})
  → Reuses existing GOAL_OPTIONS (full_set, format_complete, count_25, count_50,
just_exploring).
  → For DISCOVERY/SOCIAL/FINANCIAL primary, 2b is skipped and goalType defaults to
null
    (user can set later).
```

Step count stays “Step 2 of 5” externally (2a/2b are internal). Steps 1 (name), 3 (log-owned), 4 (formats), 5 (complete) are unchanged.

1.4 Database schema changes (Prisma)

The existing `CollectionGoalV2` model stays for completion goals. Add a new `UserMotivation` model and one denormalized field on `User` for fast personalization reads.

```
// NEW — captures the "why" (1.1 taxonomy). Separate from completion goals.
model UserMotivation {
  id          String   @id @default(uuid())
  userId      String
  user        User     @relation(fields: [userId], references: [id], onDelete:
Cascade)

  category    String   // "INVENTORY" | "SOCIAL" | "ACQUISITION" | "FINANCIAL" | "DIS-
COVERY"
  isPrimary   Boolean  @default(false)
  rank        Int      @default(0) // ordering of secondary motivations

  createdAt   DateTime @default(now())

  @@unique([userId, category])
  @@index([userId])
  @@index([userId, isPrimary])
}

// EXTEND existing User model — denormalized primary for cheap dashboard branching.
model User {
  // ... existing fields ...
  primaryMotivation String? // mirrors UserMotivation.isPrimary; null until
onboarding 2a done
  motivations       UserMotivation[]
}
```

Migration: `add_user_motivations` — creates `UserMotivation` table + adds nullable `User.primaryMotivation`. Non-breaking (nullable, default null).

1.5 Dashboard personalization rules

Driven by `User.primaryMotivation`. Implemented in `app.cabinet._index.jsx` loader + a small `getDashboardLayout(primaryMotivation)` helper in a new `app/lib/personalization.server.js`.

primaryMotivation	Default landing view	Hero widget	Secondary widgets	Notification defaults
`INVENTORY`	Collection grid	"Your Collection: X/118" with periodic table	Recent additions, format breakdown	milestone:on, restock:off
`ACQUISITION`	Missing/Shop	"Next Best Acquisitions" recommendations	Closest-to-completion rings, wishlist	milestone:on, restock:on, near-completion:on
`FINANCIAL`	Collection grid (value mode)	"Portfolio Value" total + cost basis	Investment over time, per-element value	price-change:on, restock:off
`SOCIAL`	Dashboard + share card	Public share link + preview	Friends' collections (future), share stats	milestone:on (shareable)
`DISCOVERY`	Minimal dashboard	"Explore the Table" prompt	Single CTA to browse; no pressure stats	all minimized (digest only)

Rules:
- **R-1.5.1** Secondary motivations additively enable widgets (e.g., SOCIAL secondary always adds the share card regardless of primary).
- **R-1.5.2** If `primaryMotivation` is null (legacy users), fall back to the current Phase 1 dashboard (no regression).
- **R-1.5.3** FINANCIAL primary requires Element Notes price capture (Module 4) — surface a one-time prompt to add prices if none exist.

1.6 API / route changes

- `onboarding.collection-goal.jsx` → split into motivation substep + conditional goal substep (same route, internal `phase` state, or a new `onboarding.motivation.jsx` preceding it). Recommended: new route `onboarding.motivation.jsx` as Step 2a, existing route becomes 2b.
- `action` writes: upsert `UserMotivation` rows, set `User.primaryMotivation`, then redirect to 2b (if applicable) or 3 (`/onboarding/log-owned`).
- New `app/lib/personalization.server.js`: `setMotivations(userId, {primary, secondary[]})`, `getDashboardLayout(primaryMotivation)`.
- Preferences route (`app.cabinet.preferences.jsx`): add a "Your goals & motivations" section to edit motivations post-onboarding.

1.7 UI component descriptions

- **MotivationCard** (Step 2a): icon + title + one-line description + a “Make primary” pill. Selected cards get the accent border (reuse `optionSelected` style). The primary card shows a filled star badge; secondaries show a check. Grid: 2 columns on desktop, 1 on mobile.
- INVENTORY — 📋 “Track my collection”
- SOCIAL — 🔗 “Share with friends”
- ACQUISITION — ➕ “Add missing items”
- FINANCIAL — 💰 “Track my investment”
- DISCOVERY — 🔍 “Just exploring”
- **PrimaryToggle**: radio-style; selecting “primary” on one card unsets it on others.
- **Continue button**: disabled until ≥1 selection; label “Continue →”.

1.8 Business logic & validation

- **V-1.8.1** Exactly one primary; 0–4 secondaries. Enforced client + server.
- **V-1.8.2** `category` must be in the canonical enum set (validate server-side).
- **V-1.8.3** Changing primary motivation later re-runs `getDashboardLayout` on next load; no data migration needed.
- **V-1.8.4** Completion goal (`CollectionGoalV2`) only required when primary ∈ {ACQUISITION}. INVENTORY may optionally set one.

1.9 Open questions / dependencies

- **OQ-1.1** Should motivations be mutually exclusive in v1 (single-select) to reduce complexity, deferring multi-select to v2? (Recommendation: ship multi-select; it’s cheap and matches reality.)
- **OQ-1.2** FINANCIAL value math depends on Module 4 price capture — sequence Module 4 before/with Module 1’s FINANCIAL path.
- **Dependency:** §4 (Element Notes) for FINANCIAL; §3 (Notifications) for per-motivation notification defaults.

---

2. Module 2 — Admin Dashboard Expansion

Three sub-modules: **(A) SKU Sync Validation**, **(B) Demand Signal Intelligence**, **(C) Collection Set Builder**. All render as child routes under the existing `app.admin.jsx` layout (consistent with `app.admin.operations.jsx` , `app.admin.analytics.jsx`).

2.A — SKU Sync Validation (Shopify ↔ eBay)

2.A.1 Objective

Detect and flag discrepancies between the Shopify catalog (source of truth for the storefront) and eBay active listings (secondary sales channel). Two discrepancy classes:
1. **Price delta** — same SKU/element priced inconsistently beyond an allowed band.
2. **Stock discrepancy** — item in stock on one channel, out/absent on the other; quantity mismatch.

2.A.2 User stories

- **US-2A.1** As an admin, I can import the latest eBay active-listings CSV and have it matched to Shopify products by SKU (fallback: element + format).
- **US-2A.2** As an admin, I see a reconciliation table of every mismatch with severity, so I can fix pricing/stock before customers see it.

- **US-2A.3** As an admin, I can set an allowed price-delta band (e.g., ±5%) and have anything outside it flagged.
- **US-2A.4** As an admin, I can mark a mismatch “acknowledged” (intentional) so it stops alerting.

2.A.3 Detection logic

```
For each Shopify product P with sku S:
  Find eBay listing E where E.sku == S  (fallback: E.element == P.elementSymbol AND
E.format == P.format)
  If no E:            flag SKU_MISSING_EBAY        (low/medium)
  If E and no P:      flag SKU_MISSING_SHOPIFY     (medium)
  priceDelta = (E.price - P.priceUsd) / P.priceUsd
  If |priceDelta| > band:   flag PRICE_DELTA (severity by magnitude: >20% high, 5–20%
medium)
  If P.inventoryQty > 0 XOR E.qtyAvailable > 0:  flag STOCK_DISCREPANCY (high)
  If both in stock but |qty diff| > threshold:    flag QTY_MISMATCH (low)
```

Severity → admin surfacing: **high** = top “Action Required” banner; **medium** = reconciliation table default filter; **low** = collapsed.

2.A.4 Database schema (Prisma)

```
// Raw imported eBay listing snapshot (one row per listing per import batch)
model EbayListing {
  id            String   @id @default(uuid())
  importBatchId String
  ebayItemId    String?
  sku           String?
  title         String
  elementSymbol String?
  format        String?
  price         Float
  currency      String   @default("USD")
  qtyAvailable  Int      @default(0)
  listingUrl    String?
  importedAt    DateTime @default(now())

  @@index([sku])
  @@index([importBatchId])
  @@index([elementSymbol])
}

model SkuSyncBatch {
  id            String   @id @default(uuid())
  source        String   @default("ebay_csv") // "ebay_csv" | "ebay_api" | "shopi-
fy_sync"
  fileName      String?
  rowsImported  Int      @default(0)
  mismatchCount Int      @default(0)
  createdBy     String
  createdAt     DateTime @default(now())
}

// One row per detected discrepancy, refreshed each reconciliation run
model SkuSyncIssue {
  id            String   @id @default(uuid())
  batchId       String
  productId     String?  // nullable when SKU_MISSING_SHOPIFY
  sku           String
  issueType     String   // PRICE_DELTA | STOCK_DISCREPANCY | QTY_MISMATCH |
SKU_MISSING_EBAY | SKU_MISSING_SHOPIFY
  severity      String   // "high" | "medium" | "low"
  shopifyPrice  Float?
  ebayPrice     Float?
  shopifyQty    Int?
  ebayQty       Int?
  priceDeltaPct Float?
  status        String   @default("open") // "open" | "acknowledged" | "resolved"
  acknowledgedBy String?
  resolvedAt    DateTime?
  detail        String?
  createdAt     DateTime @default(now())

  @@index([batchId])
  @@index([status])
  @@index([issueType])
  @@index([severity])
}

// Admin-configurable thresholds (single row, or per-format rows)
model SkuSyncConfig {
  id                  String  @id @default(uuid())
  priceBandPct        Float   @default(0.05) // ±5%
  qtyMismatchThreshold Int    @default(2)
```

```
  autoFlagMissing    Boolean @default(true)
  updatedBy          String?
  updatedAt          DateTime @updatedAt
}
```

2.A.5 API endpoints / routes

- `app/routes/app.admin.sync.jsx` (loader: latest batch + open issues; action: run reconciliation, acknowledge/resolve issue, save config).
- `app/routes/app.admin.sync.import.jsx` (action: multipart CSV upload → parse → write `EbayListing` + `SkuSyncBatch`, then trigger reconcile).
- Server lib `app/lib/sku-sync.server.js`: `parseEbayCsv(buffer)`, `reconcile(batchId, config)`, `acknowledgeIssue(id, admin)`, `resolveIssue(id, admin)`.

2.A.6 Admin workflow

1. Admin uploads weekly eBay CSV → batch created, listings stored.
2. System auto-runs `reconcile()` against current Shopify catalog → issues written.
3. Dashboard shows: counts by severity, "Action Required" (high) at top, filterable table (SKU, element, type, Shopify vs eBay price/qty, delta%, status).
4. Admin fixes in Shopify/eBay externally, then marks issue **resolved**, or **acknowledged** if intentional.
5. Audit: every status change records `acknowledgedBy` / `resolvedAt`.

2.A.7 Validation rules

- **V-2A.1** Import rejects rows with no price; logs them as skipped (not a product mismatch).
- **V-2A.2** SKU match is exact first; element+format fallback only when SKU absent/blank on eBay side.
- **V-2A.3** Reconciliation is idempotent per batch — re-running replaces that batch's issues, never duplicates.

2.B — Demand Signal Intelligence

2.B.1 Objective

Quantify latent demand to prioritize restocks and acquisitions. Combine wishlist counts, waitlist joins, "wanted" collection states, and out-of-stock view traffic into a **Restock Priority Score** per element/product.

2.B.2 User stories

- **US-2B.1** As an admin, I see the "Most Wanted" elements ranked by aggregate demand.
- **US-2B.2** As an admin, I see a Restock Priority list that weights demand against current stock (0 stock + high demand = top).
- **US-2B.3** As an admin, I can drill into an element to see the demand breakdown (wishlist vs waitlist vs wanted-state).

2.B.3 Scoring model

```
RestockPriorityScore(product) =
    w1 * wishlistCount            // WishlistItem rows for this product/element
  + w2 * waitlistCount            // restock-notify opt-ins (Wish-
listItem.notifyOnRestock)
  + w3 * wantedStateCount         // CollectionItem.state == "WANTED" for this element
  + w4 * recentOOSViews           // analytics: views of OOS product page (rolling
30d)
  - w5 * inventoryQty             // dampen if already in stock
Default weights: w1=3, w2=5, w3=2, w4=0.5, w5=4  (configurable, see DemandConfig)
Normalize to 0–100 for display.
```

“Most Wanted” = sort by `(wishlistCount + wantedStateCount + waitlistCount)` regardless of stock.
“Restock Priority” = sort by `RestockPriorityScore` (stock-aware).

2.B.4 Database schema (Prisma)

Most signals already exist (`WishlistItem` , `WishlistItem.notifyOnRestock` , `CollectionItem.state`).
Add a lightweight event table for view/waitlist tracking and a cached snapshot.

```
// Append-only demand events (lightweight analytics)
model DemandSignal {
  id            String   @id @default(uuid())
  elementSymbol String
  productId     String?
  signalType    String   // "wishlist_add" | "waitlist_join" | "wanted_state" | "oos_v
iew" | "search_miss"
  userId        String?
  customerId    String?
  weight        Float    @default(1)
  createdAt     DateTime @default(now())

  @@index([elementSymbol])
  @@index([signalType])
  @@index([createdAt])
}

// Cached, recomputed nightly or on-demand for the dashboard
model DemandScoreSnapshot {
  id                  String   @id @default(uuid())
  elementSymbol       String
  productId           String?
  wishlistCount       Int      @default(0)
  waitlistCount       Int      @default(0)
  wantedStateCount    Int      @default(0)
  oosViews30d         Int      @default(0)
  inventoryQty        Int      @default(0)
  restockPriorityScore Float   @default(0)
  computedAt          DateTime @default(now())

  @@unique([elementSymbol])
  @@index([restockPriorityScore])
}

model DemandConfig {
  id        String  @id @default(uuid())
  w1Wishlist Float  @default(3)
  w2Waitlist Float  @default(5)
  w3Wanted   Float  @default(2)
  w4OosView  Float  @default(0.5)
  w5Stock    Float  @default(4)
  updatedBy  String?
  updatedAt  DateTime @updatedAt
}
```

2.B.5 API / routes

- `app/routes/app.admin.demand.jsx` (already referenced in `AppNav` ADMIN_NAV) — loader returns Most Wanted + Restock Priority tables from `DemandScoreSnapshot`; action recomputes and saves weights.
- Server lib `app/lib/demand.server.js`: `recordSignal({...})`, `computeSnapshots(config)`, `getMostWanted(limit)`, `getRestockPriority(limit)`.
- Instrument existing flows to call `recordSignal`: wishlist add, waitlist join (OOS “Join Waitlist”), state→WANTED, OOS product view.

2.B.6 Admin workflow & UI

- **DemandDashboard**: two ranked tables (Most Wanted, Restock Priority), each row = element badge + symbol + score bar + component breakdown tooltip + current stock + “Mark for restock” action that creates an `AssignmentException`/purchasing note.

- Weight tuning panel (collapsible) with “Recompute” button.

2.B.7 Validation

- **V-2B.1** Snapshots are recomputed atomically (compute → upsert) to avoid partial reads.
- **V-2B.2** `oos_view` signals are de-duplicated per user per product per day to prevent inflation.

2.C — Collection Set Builder

2.C.1 Objective

Let admins curate themed sets (e.g., “Noble Gases Complete”, “Transition Metals Starter”) that power onboarding suggestions, dashboard goals, milestones, and (optionally) `CollectorPack` creation.

2.C.2 User stories

- **US-2C.1** As an admin, I can create a named set with a description, theme, and an ordered list of elements (by symbol) or SKUs.
- **US-2C.2** As an admin, I can build a set from a rule (e.g., “all elements in periodic group = noble_gases”) or hand-pick.
- **US-2C.3** As an admin, I can publish/unpublish a set; published sets appear as suggested goals to customers.
- **US-2C.4** As an admin, I can convert a set into a `CollectorPack` for subscription/store sale.

2.C.3 Database schema (Prisma)

```
model CollectionSet {
  id            String   @id @default(uuid())
  slug          String   @unique
  name          String   // "Noble Gases Complete"
  description   String   @default("")
  theme         String?  // "group" | "format" | "rarity" | "curated"
  iconEmoji     String   @default("🧪")
  ruleType      String   @default("manual") // "manual" | "group" | "format" | "rar-
ity"
  ruleValue     String?  // e.g. "noble_gases" | "lucite" | "rare"
  isPublished   Boolean  @default(false)
  sortOrder     Int      @default(0)
  createdBy     String
  createdAt     DateTime @default(now())
  updatedAt     DateTime @updatedAt

  items         CollectionSetItem[]

  @@index([isPublished])
  @@index([theme])
}

model CollectionSetItem {
  id            String   @id @default(uuid())
  setId         String
  set           CollectionSet @relation(fields: [setId], references: [id], onDelete: C
ascade)
  elementSymbol String
  sku           String?  // optional specific product
  position      Int      @default(0)

  @@unique([setId, elementSymbol])
  @@index([setId])
}
```

2.C.4 API / routes

- `app/routes/app.admin.sets.jsx` (list + create), `app/routes/app.admin.sets.$setId.jsx` (edit items, publish, convert-to-pack).
- Server lib `app/lib/collection-sets.server.js`: `createSet()`, `materializeRule(ruleType, ruleValue)` → element list, `publishSet()`, `convertToPack(setId)` → writes a `CollectorPack`.

2.C.5 Admin workflow & UI

- **SetBuilder**: left panel = set metadata (name, theme, icon, rule); right panel = element picker rendered as a mini periodic table (reuse `PeriodicTable` in a "select" mode) + ordered list with drag-to-reorder. Rule-based sets auto-fill the picker and allow manual tweaks.
- "Publish" toggles visibility to customers (feeds Module 1 ACQUISITION goal suggestions and milestone engine "set_complete" detection).

2.C.6 Validation

- **V-2C.1** A set needs ≥2 items to publish.
- **V-2C.2** `slug` unique; auto-derive from name, allow override.
- **V-2C.3** Converting to pack requires every item to map to an active SKU; otherwise list the unmapped elements and block.

2.C.7 Open questions

- **OQ-2.1** Should "set complete" be a first-class `Milestone.type` (e.g., `set_complete:<slug>`)? Recommendation: yes — extend the milestone engine to scan published sets.
- **OQ-2.2** eBay reconciliation (2.A) live API vs CSV — see OQ-0.2.

---

3. Module 3 — Notification System

Builds on the existing `notifications.server.js` dispatchers and the `NotificationLog` Prisma model. Adds preferences, an in-app inbox, and a strict trigger taxonomy.

3.1 Objective

Turn console-stub notifications into a real, preference-aware system delivering **in-app** (always) and **email** (opt-in) messages for a curated set of high-signal events.

3.2 User stories

- **US-3.1** As a collector, I get notified when I unlock a milestone.
- **US-3.2** As a collector, I get an urgency nudge when I'm ≥80% complete in a group/format/set.
- **US-3.3** As a collector, I get alerted when a wishlist item I flagged for restock is back in stock.
- **US-3.4** As a user, I control which categories I receive and through which channel.
- **US-3.5** As a user, I see an in-app notification inbox with read/unread state.
- **US-3.6** As an admin, I receive operational alerts (high-discount, assignment exceptions, sync issues) without spamming customers.

3.3 Trigger taxonomy

Event key	Trigger condition	Signal class	Default channels	Audience
`milestone_unlocked`	New `Milestone` row created	HIGH	in-app + email	customer
`near_completion`	Group/format/set crosses ≥80% (and not yet 100%)	HIGH	in-app + email	customer
`set_completed`	A published `CollectionSet` reaches 100%	HIGH	in-app + email	customer
`wishlist_restock`	`WishlistItem.notifyOnRestock` element transitions 0→>0 stock	HIGH	in-app + email	customer
`next_best_recommendation`	Weekly: new high-fit acquisition available	LOW	in-app only (digest)	customer
`price_change`	Grandfathered price changing	HIGH	in-app + email	subscriber
`shipment_assigned` / `shipment_shipped`	Subscription lifecycle	MEDIUM	in-app + email	subscriber
`high_discount_alert`	Assignment discount >20%	HIGH	email	admin
`assignment_exception`	Engine raises exception	HIGH	in-app(admin) + email	admin
`sku_sync_high`	New high-severity `SkuSyncIssue`	HIGH	in-app(admin)	admin

3.4 Delivery mechanisms

- **In-app:** new `Notification` rows rendered in a bell/inbox UI (header badge + dropdown + full page). Always created for in-app events regardless of email opt-in.

- **Email:** routed through the existing `sendEmail` dispatcher (stub now; SendGrid/Postmark in production). Gated by `NotificationPreference`.
- **SMS:** out of scope for v1 (schema leaves room via channel field); `communicationSms` already exists on `CustomerPreference`.
- **Digest:** LOW-signal events are batched into a weekly digest, never sent individually.

3.5 High-signal vs low-signal business rules

- **R-3.5.1 Signal budget:** at most **1 email per category per 24h** per user (collapse duplicates).
- **R-3.5.2 No-spam on bulk imports:** if many elements are added at once (e.g., onboarding log-owned, CSV import), milestone/near-completion notifications are **suppressed** during the batch and a single summary is emitted.
- **R-3.5.3 Near-completion fires once** per threshold crossing per group/format/set (track in `NotificationPreference`/dedupe key), not on every subsequent add.
- **R-3.5.4 DISCOVERY motivation** (Module 1) defaults all categories to digest/in-app only.
- **R-3.5.5 Admin events never go to customers** and vice versa (audience field enforced).

3.6 Database schema (Prisma)

`NotificationLog` already exists for the email/webhook audit trail. Add an **in-app** `Notification` model and a **`NotificationPreference`** model. (Note: `CustomerPreference` has coarse email/SMS toggles; the new per-category model supersedes them for granular control and can read defaults from it.)

```
// In-app notification inbox item
model Notification {
  id            String   @id @default(uuid())
  userId        String?  // customer-facing (User identity)
  customerId    String?  // when tied to Customer/subscription identity
  audience      String   @default("customer") // "customer" | "admin"
  category      String   // matches trigger taxonomy event key
  signalClass   String   @default("HIGH") // "HIGH" | "MEDIUM" | "LOW"
  title         String
  body          String
  icon          String   @default("🔔")
  linkUrl       String?  // deep link (e.g., /app/cabinet/collection?el=Ne)
  isRead        Boolean  @default(false)
  dedupeKey     String?  // e.g., "near_completion:noble_gases:80" for once-only fir-
ing
  createdAt     DateTime @default(now())
  readAt        DateTime?

  @@index([userId, isRead])
  @@index([customerId])
  @@index([category])
  @@unique([userId, dedupeKey])
}

// Per-user, per-category channel preferences
model NotificationPreference {
  id            String   @id @default(uuid())
  userId        String
  category      String   // trigger taxonomy event key, or "ALL"
  inApp         Boolean  @default(true)
  email         Boolean  @default(true)
  digest        Boolean  @default(false) // batch instead of immediate
  updatedAt     DateTime @updatedAt

  @@unique([userId, category])
  @@index([userId])
}
```

Migration `add_notification_inbox_and_prefs` — both additive. Seed default preference rows lazily (create-on-first-read) to avoid backfill.

3.7 API / routes & server lib

- `app/routes/app.cabinet.notifications.jsx` — inbox page (loader: paginated `Notification`; action: mark read / mark all read).
- Header bell component reads unread count via a small fetcher route or root loader.
- Preferences UI added to `app.cabinet.preferences.jsx` — per-category toggles (in-app/email/digest).
- Extend `app/lib/notifications.server.js`:
- `notify(userId, {category, title, body, linkUrl, dedupeKey, signalClass})` — central entry: checks prefs, applies signal budget (R-3.5.1), writes `Notification` (in-app), and conditionally calls `sendEmail`.
- `runWeeklyDigest()` — batches LOW + digest-flagged items.
- Hook points: milestone engine (`milestone_unlocked`), progress calc (`near_completion` / `set_completed`), inventory/restock (`wishlist_restock`), assignment engine (`high_discount_alert`, `assignment_exception`), sku-sync (`sku_sync_high`).

3.8 UI component descriptions

- **NotificationBell**: header icon with unread count badge; click opens a dropdown of the 5 most recent (icon, title, relative time, unread dot); “View all” → inbox page.
- **NotificationInbox**: list grouped by date; filter chips (All / Unread / Milestones / Restocks); each row links to its `linkUrl`; “Mark all read”.
- **NotificationPrefs**: table of categories × channels (in-app/email/digest) toggles; “high-signal” categories visually grouped above “low-signal”.

3.9 Open questions / dependencies

- **OQ-3.1** Real email provider choice (SendGrid vs Postmark) — deferred to production phase; stub stays for Phase 2.
- **OQ-3.2** Does the bell/unread count belong in `AppNav` or a top header? (AppNav is a sidebar; recommend a small top-right header element on cabinet routes.)
- **Dependency:** Module 2.B (`wishlist_restock` relies on inventory transitions + demand signals); Module 2.C (`set_completed`); Module 4 doesn’t block this.

---

4. Module 4 — Open Wishlist & Element Notes

4.1 Objective

1. **Open Wishlist:** add intent **context labels** (Core, Exploration, Aspirational) and filtering so the wishlist communicates why an item is wanted.
2. **Element Notes:** private collector logs on **owned** items — acquisition date, source, price paid, condition, and free-text notes — powering provenance and the FINANCIAL motivation (Module 1).

4.2 User stories

- **US-4.1** As a collector, I can label each wishlist item Core / Exploration / Aspirational.
- **US-4.2** As a collector, I can filter and sort my wishlist by label and priority.
- **US-4.3** As a collector, I can record private details for an owned element: acquisition date, source, price paid, condition, and notes.
- **US-4.4** As a collector, I see my total invested amount (sum of prices) and per-element value on the collection/portfolio view.
- **US-4.5** As a collector, my notes are private by default and never exposed on public shares.

4.3 Context labels (Wishlist)

Label	Meaning	Default sort weight
`CORE`	Definitely want; on the critical path to my goal	1 (top)
`EXPLORATION`	Curious / maybe; testing interest	2
`ASPIRATIONAL`	Dream/expensive items; someday	3

Filtering: chips for All / Core / Exploration / Aspirational on the wishlist page; combine with existing priority ordering.

4.4 Element Notes fields

Field	Type	Notes
acquisitionDate	date	defaults to today; editable
source	string	e.g., “Luciteria”, “eBay seller X”, “trade”, “gift”
pricePaid	float	optional; drives FINANCIAL totals
currency	string	default “USD”
condition	enum	`MINT` / `EXCELLENT` / `GOOD` / `FAIR` / `DAMAGED`
notes	text	free-form private notes

4.5 Database schema (Prisma)

`CollectionItem` already has a `notes` field and `acquiredDate` / `acquiredVia`. To avoid overloading a single text field and to support structured FINANCIAL data, add a dedicated `ElementNote` (1:1 with an owned `CollectionItem`), and add `contextLabel` to wishlist.

```
// EXTEND existing WishlistItem (Customer-side) OR UserWishlistElement (User-side).
// Phase 1 customer wishlist uses WishlistItem; add contextLabel there:
model WishlistItem {
  // ... existing fields (customerId, productId, priority, notifyOnRestock, addedAt) .
..
  contextLabel String @default("CORE") // "CORE" | "EXPLORATION" | "ASPIRATIONAL"
}

// If user-side wishlist (UserWishlistElement) is the active surface, mirror there:
model UserWishlistElement {
  // ... existing fields ...
  contextLabel String @default("CORE")
}

// Structured private log for an owned element (1:1 with a CollectionItem in OWNED sta
te)
model ElementNote {
  id               String   @id @default(uuid())
  collectionItemId String   @unique
  collectionItem   CollectionItem @relation(fields: [collectionItemId], references: [i
d], onDelete: Cascade)

  acquisitionDate  DateTime?
  source           String?
  pricePaid        Float?
  currency         String   @default("USD")
  condition        String?  // "MINT" | "EXCELLENT" | "GOOD" | "FAIR" | "DAMAGED"
  notes            String?  // free-form, private

  createdAt        DateTime @default(now())
  updatedAt        DateTime @updatedAt

  @@index([collectionItemId])
}

// Add back-relation on CollectionItem
model CollectionItem {
  // ... existing fields ...
  elementNote ElementNote?
}
```

Migration `add_wishlist_labels_and_element_notes` — additive (`contextLabel` has default; `ElementNote` new table).

> **Design note:** We keep `CollectionItem.notes` (quick inline note) AND `ElementNote` (structured log). The inline `notes` mirrors `ElementNote.notes` for backward compatibility; the loader prefers `ElementNote` when present.

4.6 API / routes & server lib

- Wishlist label updates: extend `app.cabinet.wishlist.jsx` action with `setLabel(itemId, label)` and a `?label=` filter in the loader.
- Element notes: extend the element-detail interaction in `app.cabinet.collection.jsx` (and the periodic-table element modal) with an action to upsert `ElementNote`.
- Server lib `app/lib/notes.server.js` : `upsertElementNote(collectionItemId, data)` , `getPortfolioValue(userId)` → sum of `pricePaid` for OWNED items.
- FINANCIAL dashboard widget (Module 1) reads `getPortfolioValue` .

4.7 UI component descriptions

- **WishlistLabelPicker**: a 3-segment control (Core / Exploration / Aspirational) on each wishlist card; color-coded chip (e.g., Core=accent, Exploration=blue, Aspirational=purple). Filter chips at the top of the wishlist page.
- **ElementNoteEditor**: shown in the element detail modal for OWNED elements — date picker (acquisition), source text, price + currency, condition dropdown, notes textarea; "Save" + "Delete note". Empty state: "Add acquisition details" CTA.
- **ElementNoteDisplay**: compact read view on the collection card back/expander: 📅 date · 🏷️ source · 💲 price · condition badge.
- **PortfolioSummary** (FINANCIAL): total invested, item count with prices, average price, most valuable element.

4.8 Business logic & validation

- **V-4.1** `contextLabel` ∈ {CORE, EXPLORATION, ASPIRATIONAL}; default CORE.
- **V-4.2** `ElementNote` only attaches to a `CollectionItem` in OWNED state; if state leaves OWNED, the note is retained but hidden (re-shown if re-owned).
- **V-4.3** `pricePaid` ≥ 0; `condition` validated against enum.
- **V-4.4** Notes are **private** — explicitly excluded from any public share payload (wishlist token / public collection). Add a test asserting share serializers omit `ElementNote` and `notes`.
- **V-4.5** Portfolio value ignores items with null `pricePaid` (counts only priced items; shows "X of Y priced").

4.9 Open questions / dependencies

- **OQ-4.1** Which wishlist model is canonical for the customer UI — `WishlistItem` (Customer) or `UserWishlistElement` (User)? Add `contextLabel` to whichever is wired to `app.cabinet.wishlist.jsx`. (See OQ-0.1.)
- **OQ-4.2** Should price history per element be tracked (revaluation over time) or just price paid? v1 = price paid only.
- **Dependency:** Module 1 FINANCIAL widget consumes `getPortfolioValue`.

---

5. Consolidated Schema Change Summary

Module	New models	Extended models	Migration name
1	`UserMotivation`	`User.primaryMotivation`	`add_user_motivations`
2.A	`EbayListing`, `SkuSyncBatch`, `SkuSyncIssue`, `SkuSyncConfig`	—	`add_sku_sync`
2.B	`DemandSignal`, `DemandScoreSnapshot`, `DemandConfig`	—	`add_demand_intelligence`
2.C	`CollectionSet`, `CollectionSetItem`	(optional `Milestone.type` set_complete)	`add_collection_sets`
3	`Notification`, `NotificationPreference`	(reuses `NotificationLog`)	`add_notification_inbox_and_prefs`
4	`ElementNote`	`WishlistItem.contextLabel` (and/or `UserWishlistElement`), `CollectionItem.elementNote`	`add_wishlist_labels_and_element_notes`

All migrations are additive (nullable columns / defaults / new tables) → **no destructive changes, safe for production rollout**.

6. Suggested Build Sequencing

1. **Module 4 (Element Notes + labels)** — unblocks FINANCIAL motivation; smallest surface.
2. **Module 1 (Enhanced Onboarding)** — depends on M4 for FINANCIAL; high product value.
3. **Module 3 (Notifications)** — depends on milestone/progress hooks; consumes M1 motivation defaults.
4. **Module 2.B (Demand)** then **2.A (SKU Sync)** then **2.C (Set Builder)** — admin tooling; 2.B feeds restock notifications in M3.

7. Risks & Cross-Module Dependencies

- **Identity duality (`User` vs `Customer`)** is the largest risk — every module touches one or both. Recommend a short spike to define a single accessor layer (`data-access.server.js`) before broad implementation. (OQ-0.1)
- **eBay data source** (CSV vs API) affects 2.A scope. (OQ-0.2)

- **Notification spam** mitigated by the signal-budget rules (§3.5); must be load-tested against bulk onboarding imports.
- **Public-share privacy** — Element Notes must be provably excluded from share payloads (V-4.4).